Assignment1 - Analysis of SVD for Fine-Tuning LLMs

資科工碩一　315551053　許顥勳

GitHub: `https://github.com/HaoHsunHsu/Big-Data`

1 Question 1

a. Report the accuracy of original Llama-3.2-1B on both GSM8K and MATH datasets.

The original Llama-3.2-1B achieves an accuracy of 0.035 on GSM8K and 0.001 on MATH (see the "Original LLM" column in Table 1).

b. Fine-tune Llama-3.2-1B by setting the target rank to 16, 64, and 256, respectively, and report the accuracy on both GSM8K and MATH datasets.

	Original LLM	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
GSM8K	0.035	0.16	0.205	0.273
MATH	0.001	0.037	0.046	0.056

Table 1: Accuracy of Llama-3.2-1B under different settings

c. Provide your observations and explanations on the performance trends with different target ranks for both datasets.

Observations. Fine-tuning improves accuracy on both datasets, with accuracy increasing across the tested target ranks. On GSM8K, accuracy rises from 0.035 for the original model to 0.160, 0.205, and 0.273 at ranks 16, 64, and 256, respectively. On MATH, accuracy increases from 0.001 to 0.037, 0.046, and 0.056. The improvement from rank 16 to 256 is larger on GSM8K (+0.113) than on MATH (+0.019).

Explanations. A larger target rank provides more trainable parameters and allows LoRA to represent more flexible weight updates. For a fixed set of target modules, the number of trainable parameters is proportional to the rank:

$$N_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}})$$

This additional flexibility can improve task adaptation. The difference between the datasets may reflect their problem difficulty. GSM8K focuses on grade-school mathematical word problems, whereas MATH contains more challenging competition-style problems requiring more advanced mathematical reasoning. Although higher ranks improve accuracy, the absolute performance remains limited, particularly on MATH, suggesting that increasing the LoRA rank alone may not be sufficient to overcome the model's reasoning limitations.

2 Question 2

a. Following Question 1, run experiments with target ranks of 16, 64, and 256, and record the training time (minute) for each setting.

Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
68.55	73.73	99.29

Table 2: Training time (minutes) for each target rank

b. Provide your observations and explanations on the training-time trends with different target ranks.

Observations. Training time increases with the target rank, but not proportionally. Going from rank 16 to 64 (a 4× increase) adds only 5.18 minutes (+7.6%), while going from rank 64 to 256 (another 4×) adds 25.56 minutes (+34.7%). Overall, rank 256 takes about 1.45× as long as rank 16.

Explanations. The number of trainable parameters grows linearly with the target rank, so a larger rank means more computation for the weight updates, more gradients, and more optimizer states. However, the cost of the forward and backward pass through the large frozen part of the model is the same for every rank, and at small ranks it dominates the total time. This is why rank 16 and rank 64 are close. At rank 256 the trainable part becomes large enough that its extra computation and memory traffic are clearly visible, so the time rises faster.

3 Question 3

a. For each dataset, perform SVD and fine-tune the model by selecting four ranges of singular values while fixing Target Rank to 16, and report the corresponding accuracy in bar charts.

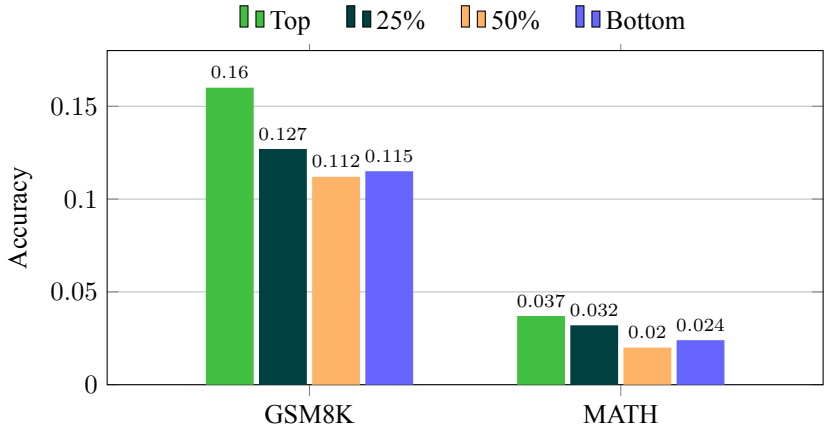


Figure 1: Accuracy for different singular-value ranges (target rank = 16).

b. Provide your observations and explanations on the performance trends for both datasets.

Observations. On both datasets, the Top 16 singular values give the best accuracy (0.160 on GSM8K, 0.037 on MATH). Accuracy then drops when the selected range moves toward smaller singular values: on GSM8K it goes 0.160 → 0.127 (25%) → 0.112 (50%), and on MATH 0.037 → 0.032 → 0.020. The Bottom 16 are slightly higher than the 50% range on both datasets (0.115 vs. 0.112 on GSM8K, 0.024 vs. 0.020 on MATH), so the trend is not strictly monotonic. The gap between Top and the worst range is 0.048 on GSM8K and 0.017 on MATH. All settings on GSM8K stay above the original model (0.035), and all on MATH stay above 0.001.

Explanations. In singular-value decomposition (SVD), larger singular values represent components that contribute more to the original weight matrix's squared Frobenius norm. Selecting these components may preserve more of the weight matrix's dominant structure, potentially providing a more suitable basis for task adaptation. In contrast, smaller singular-value components contribute less to the matrix's overall energy, which may help explain the lower accuracy observed for those ranges. The slight advantage of the Bottom range over the 50% range is small, so it should not be interpreted as a meaningful performance difference. Overall, the results suggest that selecting higher singular-value components generally produces better performance for both datasets, although the absolute accuracy on MATH remains low.

4 Question 4

a. Perform SVD on the query projection ("`q_proj`") weights of the LLM layers 0, 7, 15.

Answer: I extracted the `q_proj` weight matrix of layers 0, 7, and 15 from the original Llama-3.2-1B and applied full SVD (`torch.linalg.svd`, `full_matrices=False`) to each. The singular values were then used to compute the cumulative energy ratio for parts b and c.

b. For each layer, please report the minimum target rank required to retain at least 50% cumulative energy.

Layer 0	Layer 7	Layer 15
62	120	84

Table 3: Minimum target rank to retain at least 50% cumulative energy.

c. Please provide the following visualization for Layer 0, 7, 15, respectively. (For clarity, visualize only the first 200 singular values):

i. Cumulative energy (%) using a line chart.

ii. Singular value magnitudes using a bar chart.

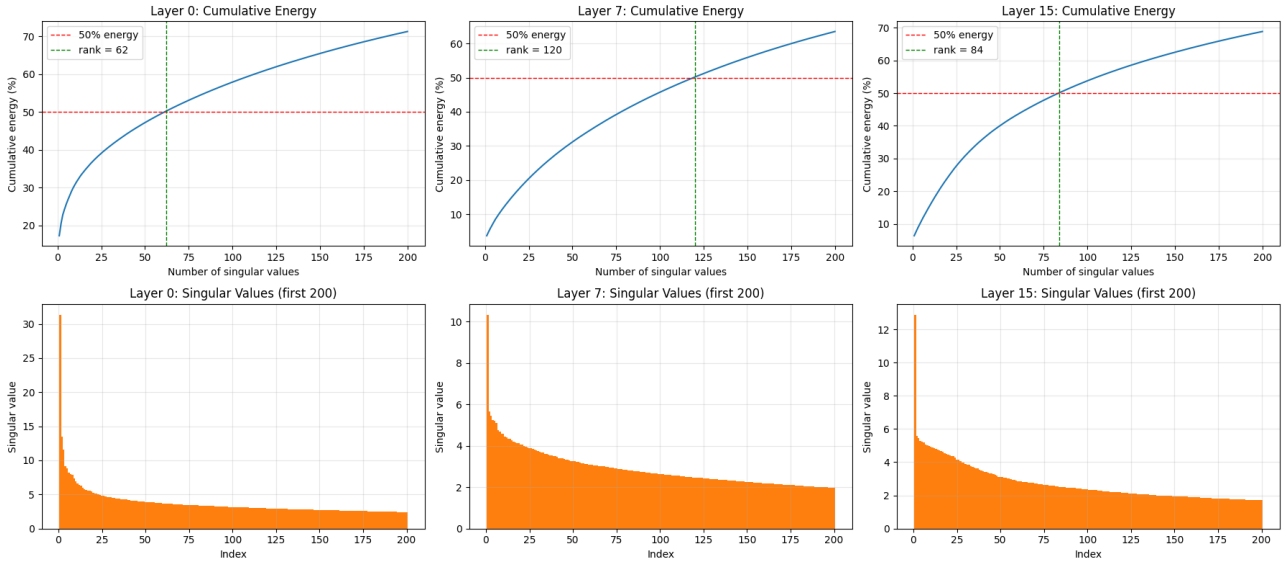


Figure 2: Cumulative energy (line) and singular value magnitude (bar) of the first 200 singular values for layers 0, 7, 15.

d. Provide your observations for the above results.

Observations. The singular-value spectra of the `q_proj` weights are highly non-uniform across all three layers, with a few dominant singular values followed by a gradually decaying tail. Layer 0 has the most concentrated spectrum: its largest singular value is approximately 31, compared with about 13 for the second-largest, and it requires the fewest components (62) to reach 50% cumulative energy. Layer 7 has a more gradual decay and requires 120 components, while Layer 15 lies between the two, requiring 84 components. Within the first 200 singular values, cumulative energy reaches approximately 71%, 64%, and 69% for Layers 0, 7, and 15, respectively.

Implications. These results show that the energy of the original `q_proj` weights is distributed across many singular components, even in Layer 0, which has the most concentrated spectrum. Since all three layers require more than 16 components to retain 50% of their energy, a rank of 16 captures less than half of the total weight energy in each layer. The differences across layers also suggest that the rank required to preserve a given proportion of weight energy varies by layer.